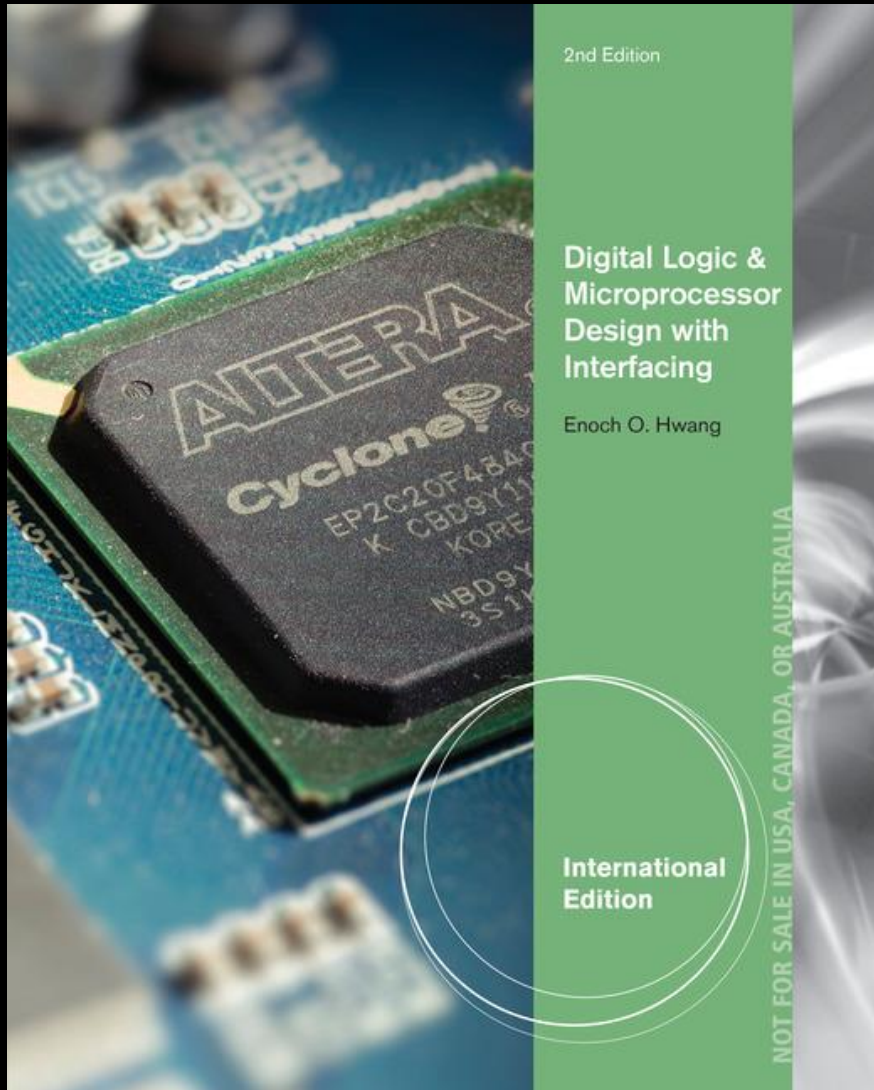




Chapter 2 Fundamentals of Digital Circuits

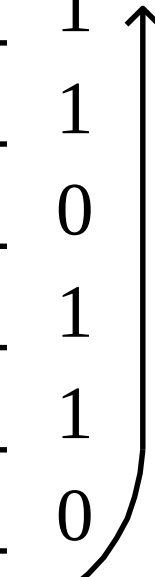
Binary Number

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Value of a Binary Number

- For decimal number:
- $658_{10} = (6 \times 10^2) + (5 \times 10^1) + (8 \times 10^0) = 600 + 50 + 8 = 658_{10}$
- For binary number:
- $1011011_2 = (1 \times 2^6) + (0 \times 2^5) + (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$
 $= 64 + 0 + 16 + 8 + 0 + 2 + 1 = 91_{10}$

Convert Decimal to Digital

2	<u>91</u>	1		Least significant bit	= 1011011
2	<u>45</u>	1			
2	<u>22</u>	0			
2	<u>11</u>	1			
2	<u>5</u>	1			
2	<u>2</u>	0			
1				Most significant bit	

Octal and Hex

■ Octal

<u>001</u>	<u>110</u>	<u>011</u>	
1	6	3	
5	7	2	4
101	111	010	100

$$\begin{aligned} 5724_8 &= (5 \times 8^3) + (7 \times 8^2) \\ &\quad + (2 \times 8^1) + (4 \times 8^0) \\ &= 2560 + 448 + 16 + 4 \\ &= 3028_{10} \end{aligned}$$

■ Hex

<u>0110</u>	<u>1101</u>	<u>1011</u>
6	D	B
5	C	F
0101	1100	1111

$$\begin{aligned} 5CA_{16} &= (5 \times 16^2) + (C \times 16^1) + \\ &\quad (F \times 16^0) \\ &= (5 \times 16^2) + (12 \times 16^1) + (15 \times \\ &\quad 16^0) \\ &= 1280 + 192 + 15 \\ &= 1487_{10} \end{aligned}$$

Base 10 to Base n Conversion

$$X_{10} = (\underbrace{\dots a_2 a_1 a_0}_I \cdot \underbrace{a_{-1} a_{-2} \dots}_{F})_n$$

$I_1 = I \div n$ 之商; $a_0 = I \div n$ 之餘

$I_2 = I_1 \div n$ 之商; $a_1 = I_1 \div n$ 之餘

$F_1 = F \times n$ 之小數; $a_{-1} = F \times n$ 之整數

$F_2 = F_1 \times n$ 之小數; $a_{-2} = F_1 \times n$ 之整數

Binary Number Arithmetic

$$\begin{array}{rcccc}
 & 1 & 0 & 0 & 1 \\
 + & 0 & 0 & 1 & 1 \\
 \hline
 & 1 & 1 & 0 & 0
 \end{array}$$

$$\begin{array}{rcccc}
 & 1 & 1 & 0 & 1 \\
 + & 1 & 0 & 1 & 1 \\
 \hline
 & 1 & 1 & 0 & 0
 \end{array}
 = \begin{array}{r} 13 \\ + 11 \\ \hline 24 \end{array}$$

$$\begin{array}{rcccc}
 & 1 & 0 & 0 & 1 \\
 - & 0 & 0 & 1 & 1 \\
 \hline
 & 0 & 1 & 1 & 0
 \end{array}
 = \begin{array}{r} 9 \\ - 3 \\ \hline 6 \end{array}$$

$$\begin{array}{rcccc}
 & 1 & 1 & 0 & 1 \\
 - & 1 & 0 & 1 & 1 \\
 \hline
 & 0 & 0 & 1 & 0
 \end{array}
 = \begin{array}{r} 13 \\ - 11 \\ \hline 2 \end{array}$$

Negative Number

- Signed or unsigned number representation
- Use two's complement representation for signed numbers
- For signed numbers, the MSB tells whether the number is positive or negative
 - 0 = positive
 - 1 = negative
- If signed number is positive then you can find the value just like for unsigned numbers

Negative Number

- If signed number is negative then you need to do two steps to find its value:
 - (1) Flip all the 1 bits to 0's and all the 0 bits to 1's.
 - (2) Add a 1 to the result obtained from step (1).
- The negated value obtained from step (2) is the value of the original signed number

1001	(original number – MSB is a 1)
(1) 0110	(flip all the bits)
(2) 0111	(add a 1 to the previous number)
0111 = 7, therefore 1001 = -7	

Negative Number

4-bit Binary	Two's Complement
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

For 4-bit **unsigned** number:
range is 0 to $2^4 - 1$
= 0 to 15

For 4-bit **signed** number:
range is -2^3 to $2^3 - 1$
= -8 to 7

For n -bit **unsigned** number:
range is 0 to $2^n - 1$

For n -bit **signed** number:
range is -2^{n-1} to $2^{n-1} - 1$

Example

- Find the two's complement number for -58

- Start with $+58$

2	58	0	Least significant bit Most significant bit
2	29	1	
2	14	0	
2	7	1	
2	3	1	
	1		

= 111010

- Binary for $+58$ is 0111010. Need to add a leading 0, otherwise it is a negative number!

0111010 = 58

1000101 Flip all the bits

1000110 add a 1 to the previous number

Therefore, 1000110 = -58

Sign Extension

- For unsigned numbers, extend with 0
- For signed numbers, extend with the MSB

	Original Number		Sign Extended		Original Number		Sign Extended
	10010		11110010		0101		00000101
Flip bits	01101		00001101				
Add 1	01110		00001110				
Value	– 14		– 14		5		5

Signed Number Arithmetic

$$\begin{array}{rcl}
 0 & 0 & 1 & 1 & = & 3 \\
 + & 1 & 0 & 0 & 1 & = & +(-7) \\
 \hline
 1 & 1 & 0 & 0 & = & -4
 \end{array}$$

$$\begin{array}{rcl}
 3 & = & 0 & 0 & 1 & 1 \\
 + (-3) & = & + & 1 & 1 & 0 & 1 \\
 \hline
 0 & = & \cancel{1} & 0 & 0 & 0 & 0
 \end{array}$$

$$\begin{array}{rcl}
 6 & = & 0 & 1 & 1 & 0 \\
 + & 3 & = & + & 0 & 0 & 1 & 1 \\
 \hline
 9 & \neq & 1 & 0 & 0 & 1
 \end{array}$$

$$\begin{array}{rcl}
 6 & = & 0 & 0 & 1 & 1 & 0 \\
 + & 3 & = & + & 0 & 0 & 0 & 1 & 1 \\
 \hline
 9 & = & 0 & 1 & 0 & 0 & 1
 \end{array}$$

Binary Arithmetic

1. Perform the following 4-bit unsigned number addition. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = \\ + & 1 & 0 & 1 & 1 & = \\ \hline & & & & & = \end{array}$$

Binary Arithmetic

1. Perform the following 4-bit unsigned number addition. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 \\ + & 1 & 0 & 1 & 1 \\ \hline 1 & 0 & 0 & 0 & 0 \end{array} \quad \begin{array}{l} = \\ = \\ = \end{array} \quad \begin{array}{l} 5 \\ +11 \\ \hline \end{array}$$

Binary Arithmetic

1. Perform the following 4-bit unsigned number addition. Is there an overflow error?

$$\begin{array}{rcccc} 0 & 1 & 0 & 1 & = & 5 \\ + & 1 & 0 & 1 & 1 & = & \underline{+11} \\ \hline 1 & 0 & 0 & 0 & 0 & = & 16 \end{array}$$

Yes, there is an overflow error

Binary Arithmetic

2. Perform the following 4-bit unsigned number addition. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = \\ + & 0 & 1 & 1 & 0 & = \\ \hline & & & & & = \end{array}$$

Binary Arithmetic

2. Perform the following 4-bit unsigned number addition. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = & & 5 \\ + & 0 & 1 & 1 & 0 & = & + & 6 \\ \hline & 1 & 0 & 1 & 1 & = & & 11 \end{array}$$

No, there is no overflow error

Binary Arithmetic

3. Perform the following 4-bit signed number addition. Is there an overflow error?

$$\begin{array}{rcccc} 0 & 1 & 0 & 1 & = \\ + & 1 & 0 & 1 & = \\ \hline & & & & = \end{array}$$

Binary Arithmetic

3. Perform the following 4-bit signed number addition. Is there an overflow error?

$$\begin{array}{rcccc} 0 & 1 & 0 & 1 & = & 5 \\ + & 1 & 0 & 1 & 1 & = & \underline{+(-5)} \\ \hline 1 & 0 & 0 & 0 & 0 & = & 0 \end{array}$$

No, there is no overflow error

Binary Arithmetic

4. Perform the following 4-bit signed number addition. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = \\ + & 0 & 1 & 1 & 0 & = \\ \hline & & & & & = \end{array}$$

Binary Arithmetic

4. Perform the following 4-bit signed number addition. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = & & 5 \\ + & 0 & 1 & 1 & 0 & = & + & 6 \\ \hline & 1 & 0 & 1 & 1 & = & & -5 \end{array}$$

Yes, there is an overflow error

Binary Arithmetic

5. Perform the following 4-bit unsigned number subtraction. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = \\ - & 0 & 1 & 1 & 0 & = \\ \hline & & & & & = \end{array}$$

Binary Arithmetic

5. Perform the following 4-bit unsigned number subtraction. Is there an overflow error?

$$\begin{array}{r} 0101 \\ - 0110 \\ \hline 1111 \end{array} = \begin{array}{r} 5 \\ - 6 \\ \hline \end{array}$$

Binary Arithmetic

5. Perform the following 4-bit unsigned number subtraction. Is there an overflow error?

$$\begin{array}{r} 0101 \\ - 0110 \\ \hline 1111 \end{array} = \begin{array}{r} 5 \\ - 6 \\ \hline 15 \end{array}$$

Yes, there is an overflow error

Binary Arithmetic

6. Perform the following 4-bit signed number subtraction. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = \\ - & 0 & 1 & 1 & 0 & = \\ \hline & & & & & = \end{array}$$

Binary Arithmetic

6. Perform the following 4-bit signed number subtraction. Is there an overflow error?

$$\begin{array}{r} 0 \ 1 \ 0 \ 1 \\ - 0 \ 1 \ 1 \ 0 \\ \hline 1 \ 1 \ 1 \ 1 \end{array} \quad = \quad \begin{array}{r} 5 \\ - 6 \\ \hline \end{array}$$

Binary Arithmetic

6. Perform the following 4-bit signed number subtraction. Is there an overflow error?

$$\begin{array}{r} 0101 \\ - 0110 \\ \hline 1111 \end{array} = \begin{array}{r} 5 \\ - 6 \\ \hline -1 \end{array}$$

No, there is no overflow error

Binary Arithmetic

7. Perform the following 4-bit signed number subtraction. Is there an overflow error?

$$\begin{array}{rcccc} & 0 & 1 & 0 & 1 & = \\ - & 1 & 0 & 0 & 0 & = \\ \hline & & & & & = \end{array}$$

Binary Arithmetic

7. Perform the following 4-bit signed number subtraction. Is there an overflow error?

$$\begin{array}{r} 0 \ 1 \ 0 \ 1 \\ - \ 1 \ 0 \ 0 \ 0 \\ \hline 1 \ 1 \ 0 \ 1 \end{array} \quad \begin{array}{l} = \\ = \\ = \end{array} \quad \begin{array}{l} 5 \\ -(-8) \\ \end{array}$$

Binary Arithmetic

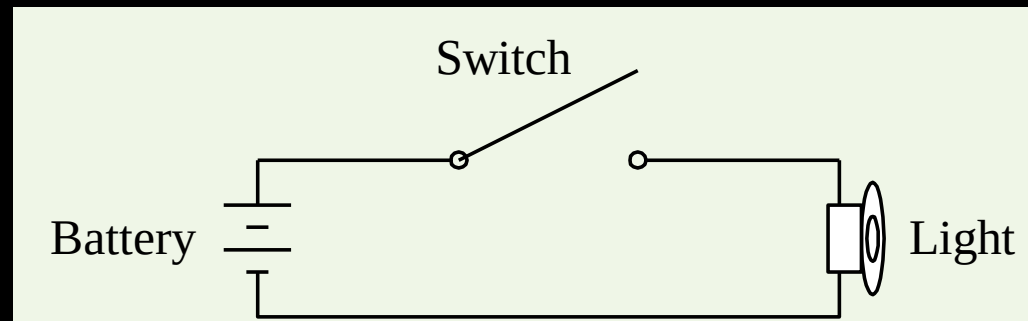
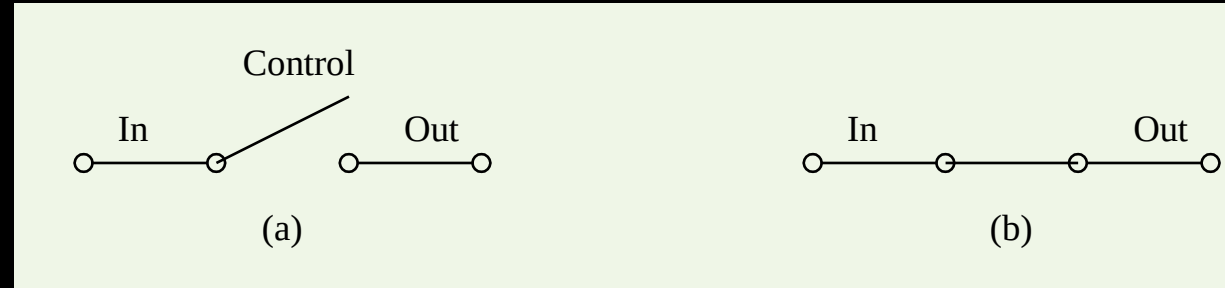
7. Perform the following 4-bit signed number subtraction. Is there an overflow error?

$$\begin{array}{r} 0101 \\ - 1000 \\ \hline 1101 \end{array} = \begin{array}{r} 5 \\ -(-8) \\ \hline -3 \end{array}$$

Yes, there is an overflow error

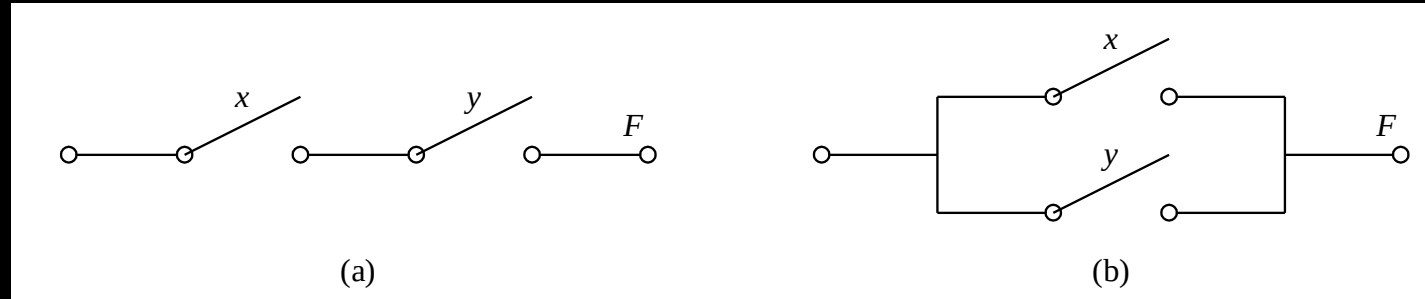
Quiz 2.1!

Binary Switch



$$L=X$$

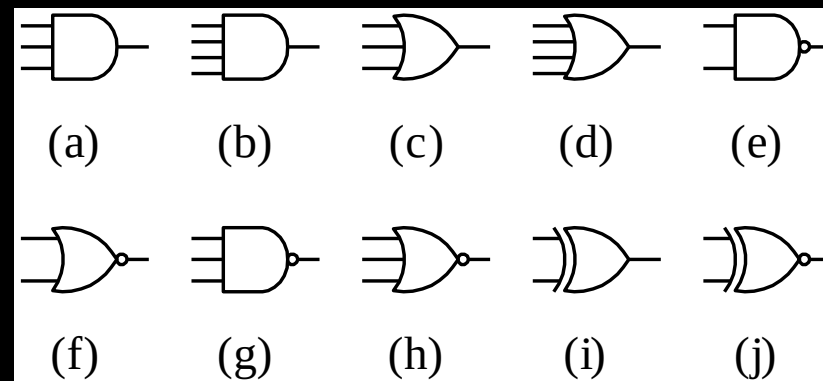
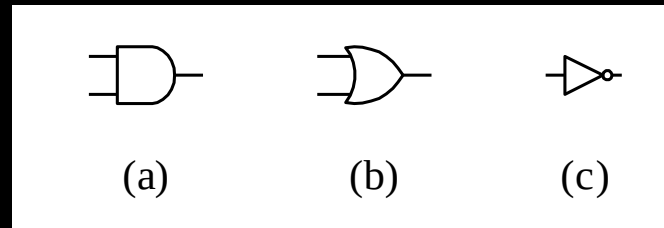
Basic Logic



- AND $F = x \text{ and } y$ $F = x \bullet y$ $F = xy$
- OR $F = x \text{ or } y$ $F = x + y$
- NOT $F = x'$
- Precedence from high to low: NOT/AND/OR
- $F = xy + z'$ $F = x(y + z)'$

Logic Gate

- **Logic gates** are the actual physical devices that implement the logical operators
- Using Logic Symbol to denote logic gates



- NAND
- NOR
- XOR
- XNOR

Logic Gate

- NAND $F = (xy)'$
- NOR $F = (x + y)'$
- XOR $F = x \oplus y = x'y + xy'$
- XNOR $F = x \odot y = x'y' + xy$

For even number of inputs $\text{xor} = \text{xnor}'$

$$(x \oplus y) = (x \odot y)'$$

For odd number of inputs $\text{xor} = \text{xnor}$

$$(x \oplus y \oplus z) = (x \odot y \odot z)$$

Truth Table

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

X	F
0	1
1	0

For even number of inputs xor = xnor'
 $(x \oplus y) = (x \odot y)'$

		2-NAND	2-NOR	2-XOR	2-XNOR
x	y	$(x \bullet y)'$	$(x + y)'$	$x \oplus y$	$x \odot y$
0	0	1	1	0	1
0	1	1	0	1	0
1	0	1	0	1	0
1	1	0	0	0	1

Truth Table

For odd number of inputs xor = xnor

$$(x \oplus y \oplus z) = (x \odot y \odot z)$$

			3-AND	3-OR	3-NAND	3-NOR	3-XOR	3-XNOR
x	y	z	$x \bullet y \bullet z$	$x + y + z$	$(x \bullet y \bullet z)'$	$(x + y + z)'$	$x \oplus y \oplus z$	$x \odot y \odot z$
0	0	0	0	0	1	1	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	0	1	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	0	1	1	0	0	0
1	1	0	0	1	1	0	0	0
1	1	1	1	1	0	0	1	1

NAND Gate

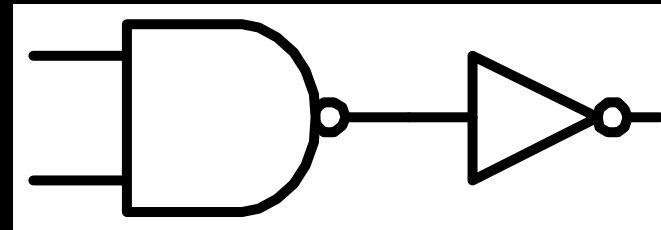
- How can you use a NAND gate to work like an AND gate?

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

NAND Gate

- How can you use a NAND gate to work like an AND gate?

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0



NAND Gate

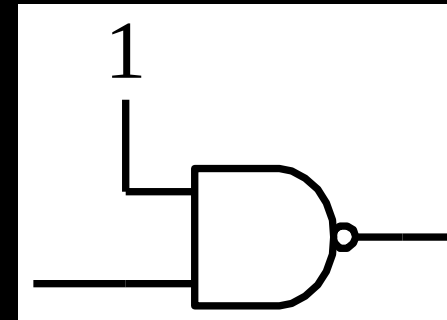
- How can you use a NAND gate to work like a NOT gate?

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

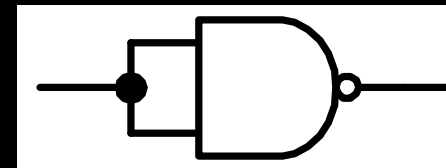
NAND Gate

- How can you use a NAND gate to work like a NOT gate?

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0



or



NAND Gate

- How can you use a NAND gate to work like an OR gate?

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

NAND Gate

- How can you use a NAND gate to work like an OR gate?

Use DeMorgan's Theorem

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

NAND Gate

- How can you use a NAND gate to work like an OR gate?

Use DeMorgan's Theorem

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

$$\begin{aligned}x+y &= (x+y)'' \\ &= \end{aligned}$$

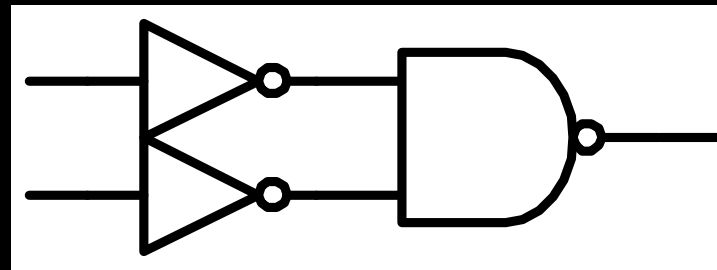
NAND Gate

- How can you use a NAND gate to work like an OR gate?

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

Use DeMorgan's Theorem

$$\begin{aligned}x + y &= (x + y)'' \\ &= (x'y')'\end{aligned}$$



XOR Gate

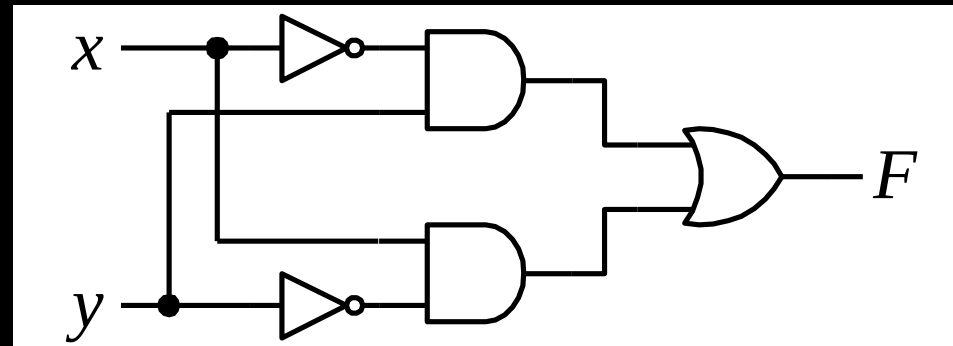
- Use AND, OR, and NOT gates to implement the XOR gate

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

XOR Gate

- Use AND, OR, and NOT gates to implement the XOR gate

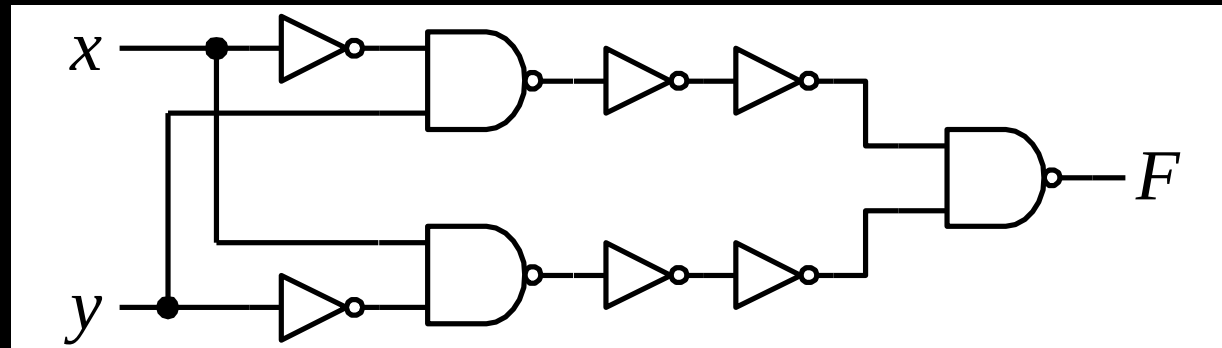
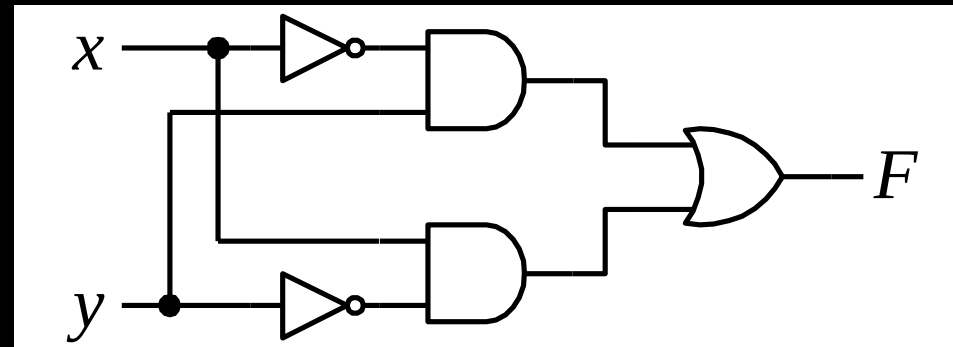
x	y	F
0	0	0
0	1	1
1	0	1
1	1	0



XOR Gate

- Use only NAND gates to implement the XOR gate

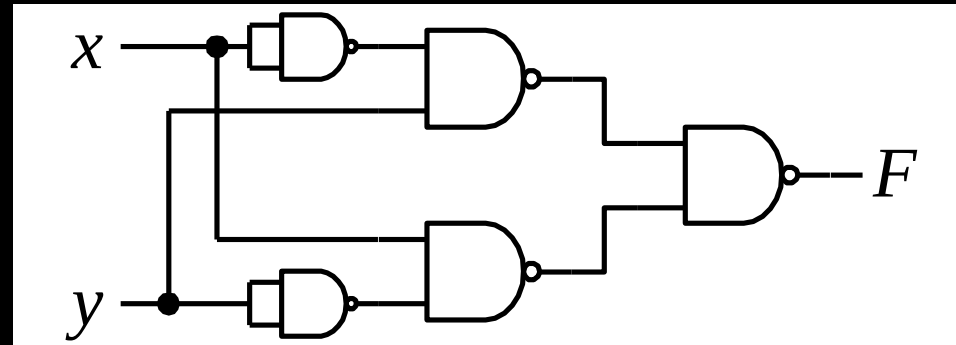
x	y	F
0	0	0
0	1	1
1	0	1
1	1	0



XOR Gate

- Use only NAND gates to implement the XOR gate

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0



Boolean Algebra

- Circuits built with binary switches can be described using Boolean algebra.
- Let $B = \{0,1\}$ be the Boolean algebra. We have axioms, single variable theorems, and two or three variable theorems.
- Can be used to reduce circuit size.

Boolean Algebra

1a.	$0 \bullet 0 = 0$	1b.	$1 + 1 = 1$	
2a.	$1 \bullet 1 = 1$	2b.	$0 + 0 = 0$	
3a.	$0 \bullet 1 = 1 \bullet 0 = 0$	3b.	$1 + 0 = 0 + 1 = 1$	
4a.	$0' = 1$	4b.	$1' = 0$	
		(a)		
5a.	$x \bullet 0 = 0$	5b.	$x + 1 = 1$	Null Element
6a.	$x \bullet 1 = 1 \bullet x = x$	6b.	$x + 0 = 0 + x = x$	Identity 統一
7a.	$x \bullet x = x$	7b.	$x + x = x$	Idempotent 冪等
8a.	$(x')' = x$			Double Complement
9a.	$x \bullet x' = 0$	9b.	$x + x' = 1$	Inverse
		(b)		
10a.	$x \bullet y = y \bullet x$	10b.	$x + y = y + x$	Commutative 交換
11a.	$(x \bullet y) \bullet z = x \bullet (y \bullet z)$	11b.	$(x + y) + z = x + (y + z)$	Associative 結合
12a.	$(x \bullet y) + (x \bullet z) = x \bullet (y + z)$	12b.	$(x + y) \bullet (x + z) = x + (y \bullet z)$	Distributive 分配
13a.	$(x \bullet y)' = x' + y'$	13b.	$(x + y)' = x' \bullet y'$	DeMorgan's
		(c)		

Reduce Logic Expression using Boolean Algebra

- Use Boolean algebra to reduce the expression $x + (x \bullet y)$ as much as possible

$$\begin{aligned}
 x + (x \bullet y) &= (x \bullet 1) + (x \bullet y) \\
 &= x \bullet (1 + y) \\
 &= x \bullet (1) \\
 &= x
 \end{aligned}$$

1a.	$0 \bullet 0 = 0$	1b.	$1 + 1 = 1$	
2a.	$1 \bullet 1 = 1$	2b.	$0 + 0 = 0$	
3a.	$0 \bullet 1 = 1 \bullet 0 = 0$	3b.	$1 + 0 = 0 + 1 = 1$	
4a.	$0' = 1$	4b.	$1' = 0$	
(a)				
5a.	$x \bullet 0 = 0$	5b.	$x + 1 = 1$	Null Element
6a.	$x \bullet 1 = 1 \bullet x = x$	6b.	$x + 0 = 0 + x = x$	Identity 統一
7a.	$x \bullet x = x$	7b.	$x + x = x$	Idempotent 冪等
8a.	$(x')' = x$			Double Complement
9a.	$x \bullet x' = 0$	9b.	$x + x' = 1$	Inverse
(b)				
10a.	$x \bullet y = y \bullet x$	10b.	$x + y = y + x$	Commutative 交換
11a.	$(x \bullet y) \bullet z = x \bullet (y \bullet z)$	11b.	$(x + y) + z = x + (y + z)$	Associative 結合
12a.	$(x \bullet y) + (x \bullet z) = x \bullet (y + z)$	12b.	$(x + y) \bullet (x + z) = x + (y \bullet z)$	Distributive 分配
13a.	$(x \bullet y)' = x' + y'$	13b.	$(x + y)' = x' \bullet y'$	DeMorgan's
(c)				

Boolean Algebra

- Use Boolean algebra to reduce the equation as much as possible

$$F = (x'yz) + (xy'z) + (xyz') + (xyz)$$

1a.	$0 \bullet 0 = 0$	1b.	$1 + 1 = 1$	
2a.	$1 \bullet 1 = 1$	2b.	$0 + 0 = 0$	
3a.	$0 \bullet 1 = 1 \bullet 0 = 0$	3b.	$1 + 0 = 0 + 1 = 1$	
4a.	$0' = 1$	4b.	$1' = 0$	
(a)				
5a.	$x \bullet 0 = 0$	5b.	$x + 1 = 1$	Null Element
6a.	$x \bullet 1 = 1 \bullet x = x$	6b.	$x + 0 = 0 + x = x$	Identity 統一
7a.	$x \bullet x = x$	7b.	$x + x = x$	Idempotent 冪等
8a.	$(x')' = x$			Double Complement
9a.	$x \bullet x' = 0$	9b.	$x + x' = 1$	Inverse
(b)				
10a.	$x \bullet y = y \bullet x$	10b.	$x + y = y + x$	Commutative 交換
11a.	$(x \bullet y) \bullet z = x \bullet (y \bullet z)$	11b.	$(x + y) + z = x + (y + z)$	Associative 結合
12a.	$(x \bullet y) + (x \bullet z) = x \bullet (y + z)$	12b.	$(x + y) \bullet (x + z) = x + (y \bullet z)$	Distributive 分配
13a.	$(x \bullet y)' = x' + y'$	13b.	$(x + y)' = x' \bullet y'$	DeMorgan's
(c)				

Boolean Algebra

- Use Boolean algebra to reduce the equation as much as possible

$$F = (x'yz) + (xy'z) + (xyz') + (xyz)$$

$$= (x'yz) + (xy'z) + (xyz') + (xyz) + (xyz) + (xyz)$$

1a.	$0 \bullet 0 = 0$	1b.	$1 + 1 = 1$	
2a.	$1 \bullet 1 = 1$	2b.	$0 + 0 = 0$	
3a.	$0 \bullet 1 = 1 \bullet 0 = 0$	3b.	$1 + 0 = 0 + 1 = 1$	
4a.	$0' = 1$	4b.	$1' = 0$	
(a)				
5a.	$x \bullet 0 = 0$	5b.	$x + 1 = 1$	Null Element
6a.	$x \bullet 1 = 1 \bullet x = x$	6b.	$x + 0 = 0 + x = x$	Identity 統一
7a.	$x \bullet x = x$	7b.	$x + x = x$	Idempotent 冪等
8a.	$(x')' = x$			Double Complement
9a.	$x \bullet x' = 0$	9b.	$x + x' = 1$	Inverse
(b)				
10a.	$x \bullet y = y \bullet x$	10b.	$x + y = y + x$	Commutative 交換
11a.	$(x \bullet y) \bullet z = x \bullet (y \bullet z)$	11b.	$(x + y) + z = x + (y + z)$	Associative 結合
12a.	$(x \bullet y) + (x \bullet z) = x \bullet (y + z)$	12b.	$(x + y) \bullet (x + z) = x + (y \bullet z)$	Distributive 分配
13a.	$(x \bullet y)' = x' + y'$	13b.	$(x + y)' = x' \bullet y'$	DeMorgan's
(c)				

Boolean Algebra

- Use Boolean algebra to reduce the equation as much as possible

$$\begin{aligned}
 F &= (x'yz) + (xy'z) + (xyz') + (xyz) \\
 &= (x'yz) + (xy'z) + (xyz') + (xyz) + (xyz) + (xyz) \\
 &= (x'yz) + (xyz) + (xy'z) + (xyz) + (xyz') + (xyz) \\
 &= [(x'yz) + (xyz)] + [(xy'z) + (xyz)] + [(xyz') + (xyz)]
 \end{aligned}$$

1a. $0 \bullet 0 = 0$	1b. $1 + 1 = 1$	
2a. $1 \bullet 1 = 1$	2b. $0 + 0 = 0$	
3a. $0 \bullet 1 = 1 \bullet 0 = 0$	3b. $1 + 0 = 0 + 1 = 1$	
4a. $0' = 1$	4b. $1' = 0$	
(a)		
5a. $x \bullet 0 = 0$	5b. $x + 1 = 1$	Null Element
6a. $x \bullet 1 = 1 \bullet x = x$	6b. $x + 0 = 0 + x = x$	Identity 統一
7a. $x \bullet x = x$	7b. $x + x = x$	Idempotent 冪等
8a. $(x')' = x$		Double Complement
9a. $x \bullet x' = 0$	9b. $x + x' = 1$	Inverse
(b)		
10a. $x \bullet y = y \bullet x$	10b. $x + y = y + x$	Commutative 交換
11a. $(x \bullet y) \bullet z = x \bullet (y \bullet z)$	11b. $(x + y) + z = x + (y + z)$	Associative 結合
12a. $(x \bullet y) + (x \bullet z) = x \bullet (y + z)$	12b. $(x + y) \bullet (x + z) = x + (y \bullet z)$	Distributive 分配
13a. $(x \bullet y)' = x' + y'$	13b. $(x + y)' = x' \bullet y'$	DeMorgan's
(c)		

Boolean Algebra

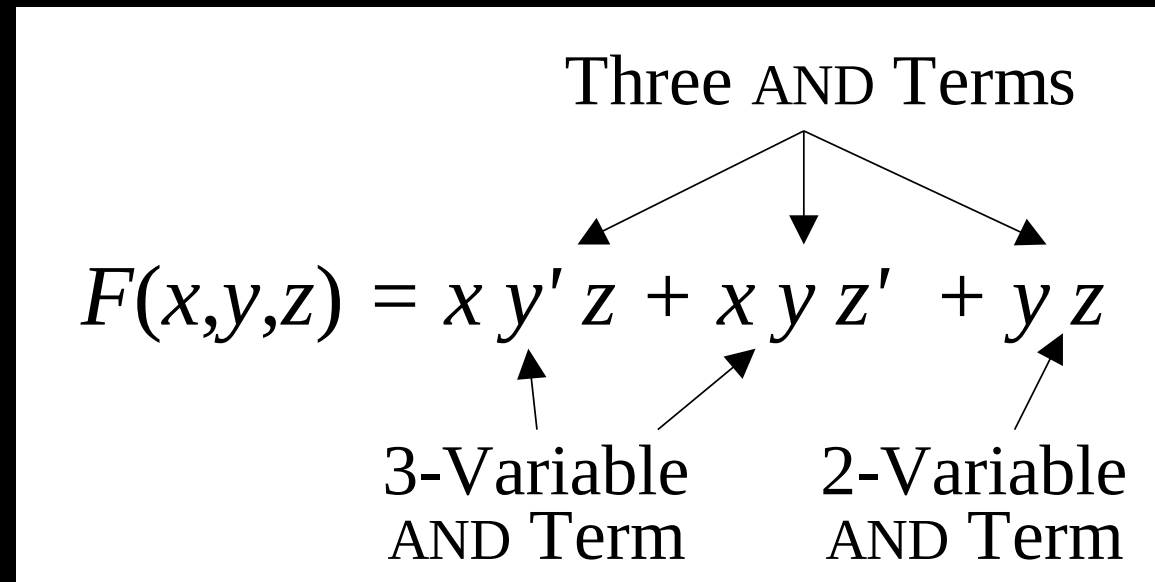
- Use Boolean algebra to reduce the equation as much as possible

$$\begin{aligned}
 F &= (x'yz) + (xy'z) + (xyz') + (xyz) \\
 &= (x'yz) + (xy'z) + (xyz') + (xyz) + (xyz) + (xyz) \\
 &= (x'yz) + (xyz) + (xy'z) + (xyz) + (xyz') + (xyz) \\
 &= [(x'yz) + (xyz)] + [(xy'z) + (xyz)] + [(xyz') + (xyz)] \\
 &= yz(x' + x) + xz(y' + y) + xy(z' + z) \\
 &= yz(1) + xz(1) + xy(1) \\
 &= yz + xz + xy \\
 &= z(y + x) + xy
 \end{aligned}$$

1a. $0 \bullet 0 = 0$	1b. $1 + 1 = 1$	
2a. $1 \bullet 1 = 1$	2b. $0 + 0 = 0$	
3a. $0 \bullet 1 = 1 \bullet 0 = 0$	3b. $1 + 0 = 0 + 1 = 1$	
4a. $0' = 1$	4b. $1' = 0$	
(a)		
5a. $x \bullet 0 = 0$	5b. $x + 1 = 1$	Null Element
6a. $x \bullet 1 = 1 \bullet x = x$	6b. $x + 0 = 0 + x = x$	Identity 統一
7a. $x \bullet x = x$	7b. $x + x = x$	Idempotent 冪等
8a. $(x')' = x$		Double Complement
9a. $x \bullet x' = 0$	9b. $x + x' = 1$	Inverse
(b)		
10a. $x \bullet y = y \bullet x$	10b. $x + y = y + x$	Commutative 交換
11a. $(x \bullet y) \bullet z = x \bullet (y \bullet z)$	11b. $(x + y) + z = x + (y + z)$	Associative 結合
12a. $(x \bullet y) + (x \bullet z) = x \bullet (y + z)$	12b. $(x + y) \bullet (x + z) = x + (y \bullet z)$	Distributive 分配
13a. $(x \bullet y)' = x' + y'$	13b. $(x + y)' = x' \bullet y'$	DeMorgan's
(c)		

Boolean Function and Inverse

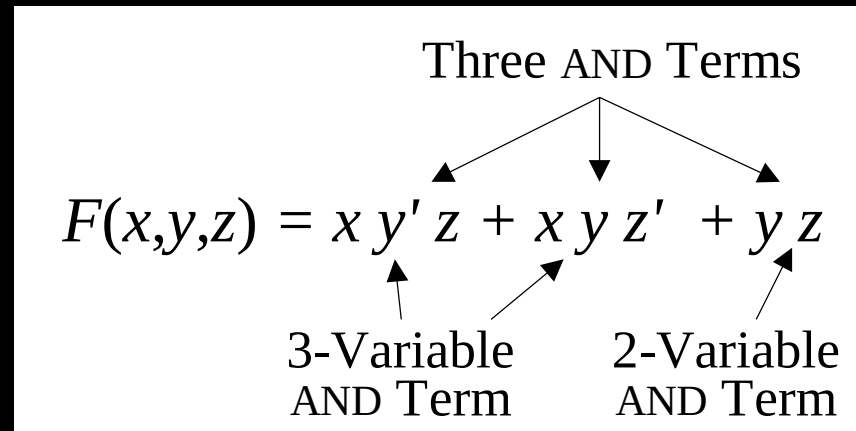
- Boolean function: logic expression to describe digital circuit.



Sum-of-product

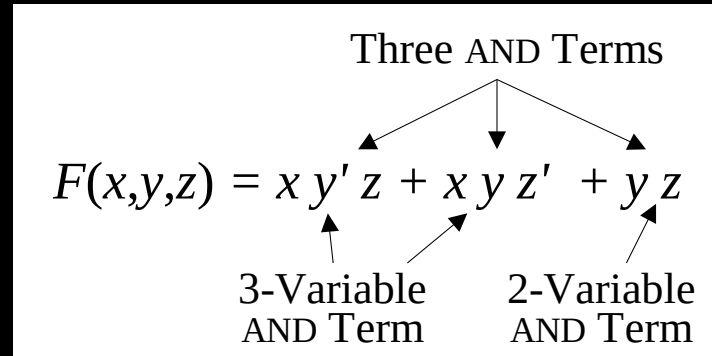
Boolean Function and Inverse

- We are mainly interested in when a function evaluates to a 1



- $F = 1$ when any one of the three AND terms evaluate to a 1
- The first AND term, $xy'z$, equals 1 if
 $x = 1, y = 0, \text{ and } z = 1$

Boolean Function and Inverse

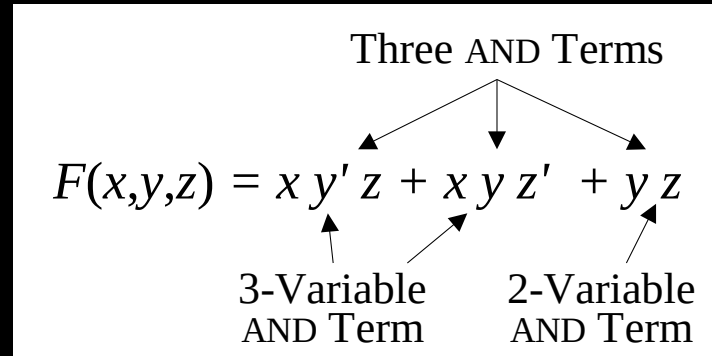


- The last AND term, yz , equals 1 if

$$y = 1 \text{ and } z = 1$$

the missing variable, x , means it doesn't matter what its value is, so it can be either 0 or 1

Boolean Function and Inverse



- Putting everything together, $F = 1$ when

$$x = 1, y = 0, \text{ and } z = 1$$

or $x = 1, y = 1, \text{ and } z = 0$

or $x = 0, y = 1, \text{ and } z = 1$

or $x = 1, y = 1, \text{ and } z = 1$

Boolean Function and Inverse

- It is more convenient to summarize this verbal description of a function with a truth table

x	y	z	F	F'
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

Boolean Function and Inverse

- The inverse of a function, F' , can be obtained easily from the truth table

x	y	z	F	F'
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

- Look at the rows where $F' = 1$
- $F' = (x'y'z') + (x'y'z) + (x'yz') + (xy'z')$

Boolean Function and Inverse

- To get F' using Boolean Algebra requires using DeMorgan's Theorem twice

$$F = xy'z + xyz' + yz$$

$$F' = (xy'z + xyz' + yz)'$$

$$= (xy'z)' \cdot (xyz')' \cdot (yz)'$$

$$= (x'+y+z') \cdot (x'+y'+z) \cdot (y'+z')$$

Boolean Function and Inverse

- We have two different equations for F'

$$F' = (x'y'z') + (x'y'z) + (x'yz') + (xy'z')$$

sum-of-products

$$F' = (x'+y+z') \cdot (x'+y'+z) \cdot (y'+z')$$

product-of-sums

Quiz 2.2!

Minterms and Maxterms

- Minterm
 - Is a product term that contains all the variables in a function
 - The variable is negated (primed) if the value is a 0
- Maxterm
 - Is a sum term that contains all the variables in a function
 - The variable is negated (primed) if the value is a 1

Minterms and Maxterms

- m_i for minterms
- M_i for maxterms where $0 \leq i < 2^n$ for n variables

x	y	z	Minterm	Notation		Maxterm	Notation
0	0	0	$x' y' z'$	m_0		$x + y + z$	M_0
0	0	1	$x' y' z$	m_1		$x + y + z'$	M_1
0	1	0	$x' y z'$	m_2		$x + y' + z$	M_2
0	1	1	$x' y z$	m_3		$x + y' + z'$	M_3
1	0	0	$x y' z'$	m_4		$x' + y + z$	M_4
1	0	1	$x y' z$	m_5		$x' + y + z'$	M_5
1	1	0	$x y z'$	m_6		$x' + y' + z$	M_6
1	1	1	$x y z$	m_7		$x' + y' + z'$	M_7

Minterm/Maxterm Example

- $F = x'y + xy' + xy$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

$$F(x, y) = m_1 + m_2 + m_3$$

$$F(x, y) = \Sigma(1, 2, 3)$$

$$F'(x, y) = \Sigma(0)$$

- $F = x'y + xy' + xy$
 $= (x'y + xy' + xy)''$
 $= \dots$

$$F(x, y) = M_0$$

$$F(x, y) = \Pi(0)$$

$$F'(x, y) = \Pi(1, 2, 3)$$

Minterm/Maxterm Example

$$\begin{aligned}
 F &= xy'z + xyz' + yz \\
 &= x'yz + xy'z + xyz' + xyz
 \end{aligned}$$

x	y	z	F	F'	Minterm	Notation
0	0	0	0	1	$x' y' z'$	m_0
0	0	1	0	1	$x' y' z$	m_1
0	1	0	0	1	$x' y z'$	m_2
0	1	1	1	0	$x' y z$	m_3
1	0	0	0	1	$x y' z'$	m_4
1	0	1	1	0	$x y' z$	m_5
1	1	0	1	0	$x y z'$	m_6
1	1	1	1	0	$x y z$	m_7

$$F(x, y, z) = m_3 + m_5 + m_6 + m_7$$

$$F(x, y, z) = \Sigma(3, 5, 6, 7)$$

$$F'(x, y, z) = \Sigma(0, 1, 2, 4)$$

$$\begin{aligned}
 F &= xy'z + xyz' + yz \\
 &= (x + y + z) \cdot (x + y + z') \cdot \\
 &\quad (x + y' + z) \cdot (x' + y + z)
 \end{aligned}$$

x	y	z	F	F'	Maxterm	Notation
0	0	0	0	1	$x + y + z$	M_0
0	0	1	0	1	$x + y + z'$	M_1
0	1	0	0	1	$x + y' + z$	M_2
0	1	1	1	0	$x + y' + z'$	M_3
1	0	0	0	1	$x' + y + z$	M_4
1	0	1	1	0	$x' + y + z'$	M_5
1	1	0	1	0	$x' + y' + z$	M_6
1	1	1	1	0	$x' + y' + z'$	M_7

$$F(x, y, z) = M_0 \cdot M_1 \cdot M_2 \cdot M_4$$

$$F(x, y, z) = \Pi(0, 1, 2, 4)$$

$$F'(x, y, z) = \Pi(3, 5, 6, 7)$$

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- Write out the full function

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- Write out the full function

$$F(x, y, z) = x'y'z + x'yz' + x'yz + x'yz' + xyz' + xyz$$

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F using Π ?

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F using Π ?
- $F(x, y, z) = \Pi(0, 4)$

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F using Π ?
- $F(x, y, z) = \Pi(0, 4)$
- Write out the full function for $\Pi(0, 4)$

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F using Π ?
- $F(x, y, z) = \Pi(0, 4)$
- Write out the full function for $\Pi(0, 4)$
- $F(x, y, z) = (x + y + z) \bullet (x' + y + z)$

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F' using Σ ?
- $F' = \Sigma(0, 4)$

Minterm/Maxterm Example

- Given $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F' using Π ?
- $F' = \Pi(1, 2, 3, 5, 6, 7)$

Minterm/Maxterm Example

- Given $F(w, x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- Write out the full function

Minterm/Maxterm Example

- Given $F(w, x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F using Π ?

Minterm/Maxterm Example

- Given $F(w, x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F using Π ?
- Write out the full function for Π of 0-Maxterms

Minterm/Maxterm Example

- Given $F(w, x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F' using Σ ?

Minterm/Maxterm Example

- Given $F(w, x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$
- What is F' using Π ?

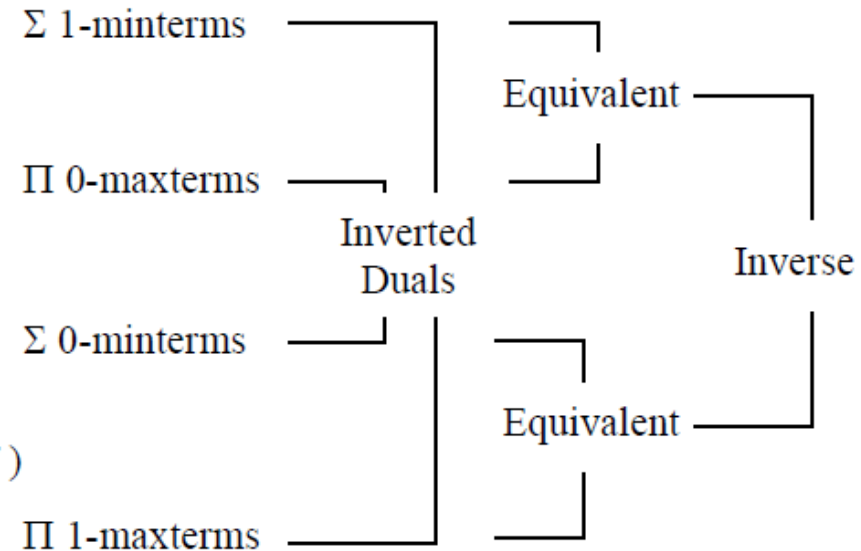
Minterms/Maxterms Relationship

$$\begin{aligned} F(x, y, z) &= x' y z + x y' z + x y z' + x y z \\ &= m_3 + m_5 + m_6 + m_7 \\ &= \Sigma(3, 5, 6, 7) \end{aligned}$$

$$\begin{aligned} &= (x+y+z) \bullet (x+y+z') \bullet (x+y'+z) \bullet (x'+y+z) \\ &= M_0 \bullet M_1 \bullet M_2 \bullet M_4 \\ &= \Pi(0, 1, 2, 4) \end{aligned}$$

$$\begin{aligned} F'(x, y, z) &= x' y' z' + x' y' z + x' y z' + x y' z' \\ &= m_0 + m_1 + m_2 + m_4 \\ &= \Sigma(0, 1, 2, 4) \end{aligned}$$

$$\begin{aligned} &= (x+y'+z') \bullet (x'+y+z') \bullet (x'+y'+z) \bullet (x'+y'+z') \\ &= M_3 \bullet M_5 \bullet M_6 \bullet M_7 \\ &= \Pi(3, 5, 6, 7) \end{aligned}$$



Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Write it in the Σ of minterms format and Π of maxterms format
- Use Truth Table

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Use Truth Table

x	y	z	F	Minterm	Notation
0	0	0		$x' y' z'$	m_0
0	0	1		$x' y' z$	m_1
0	1	0		$x' y z'$	m_2
0	1	1		$x' y z$	m_3
1	0	0		$x y' z'$	m_4
1	0	1		$x y' z$	m_5
1	1	0		$x y z'$	m_6
1	1	1		$x y z$	m_7

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Use Truth Table

x	y	z	F	Minterm	Notation
0	0	0	0	$x' y' z'$	m_0
0	0	1	1	$x' y' z$	m_1
0	1	0	1	$x' y z'$	m_2
0	1	1	1	$x' y z$	m_3
1	0	0	0	$x y' z'$	m_4
1	0	1	0	$x y' z$	m_5
1	1	0	1	$x y z'$	m_6
1	1	1	1	$x y z$	m_7

$$F = \Sigma(1, 2, 3, 6, 7) \text{ and } F = \Pi(0, 4, 5)$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Write it in the Σ of minterms format
- Use Boolean algebra

$$F = y + x'z$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Write it in the Σ of minterms format
- Use Boolean algebra

$$\begin{aligned} F &= y + x'z \\ &= y(x+x')(z+z') + x'z(y+y') \\ &= xyz + xyz' + x'yz + x'yz' + \cancel{x'yz} + x'y'z \\ &= m_7 + m_6 + m_3 + m_2 + m_1 \\ &= \Sigma(1, 2, 3, 6, 7) \end{aligned}$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Write it in the Π of maxterms format
- Use Boolean algebra

$$F = y + x'z$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Write it in the Π of maxterms format
- Use Boolean algebra

$$\begin{aligned} F &= y + x'z \\ &= (y+x')(y+z) \\ &= (y+x'+zz')(y+z+xx') \\ &= (x'+y+z)(x'+y+z')(x+y+z)(\cancel{x'+y+z}) \\ &= M_4 \cdot M_5 \cdot M_0 = \Pi(0, 4, 5) \end{aligned}$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Write F' in the Σ of minterms format
- Use Boolean algebra

$$F' = (y + x'z)'$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$

$$\begin{aligned} F' &= (y + x'z)' \\ &= y' \cdot (x'z)' \\ &= y' \cdot (x + z') \\ &= y'x + y'z' \\ &= y'x(z + z') + y'z'(x + x') \\ &= xy'z + xy'z' + \cancel{xy'z'} + x'y'z' \\ &= m_5 + m_4 + m_0 \\ &= \Sigma (0, 4, 5) \end{aligned}$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$
- Write F' in the Π of maxterms format
- Use Boolean algebra

$$F' = (y + x'z)'$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$

$$\begin{aligned} F' &= (y + x'z)' \\ &= y' \cdot (x'z)' \\ &= y' \cdot (x + z') \\ &= (y' + xx' + zz') \cdot (x + z' + yy') \\ &= (x + y' + z) (x + y' + z') (x' + y' + z) (x' + y' + z') \\ &\quad (x + y + z') (\cancel{x + y' + z'}) \\ &= M_2 \cdot M_3 \cdot M_6 \cdot M_7 \cdot M_1 \\ &= \Pi(1, 2, 3, 6, 7) \end{aligned}$$

Converting to Minterms/Maxterms

- Given $F(x, y, z) = y + x'z$

$$F = \Sigma(1, 2, 3, 6, 7) = \Pi(0, 4, 5)$$

$$F' = \Sigma(0, 4, 5) = \Pi(1, 2, 3, 6, 7)$$

Canonical, Standard, and Non-standard Forms

- Canonical: Boolean function expressed in sum-of-minterms or product-of-maxterms

$$F = x' y z + x y' z + x y z' + x y z$$

$$F' = (x+y'+z') \cdot (x'+y+z') \cdot (x'+y'+z) \cdot (x'+y'+z')$$

$$F_1(x, y, z) = \Sigma(0, 1, 2, 3, 4, 5) \quad F_2(x, y, z) = \Pi(6, 7)$$

$$F_1(x, y, z) = \Sigma(3, 5, 6) \quad F_2(x, y, z) = \Pi(3, 5, 6)$$

- Standard: sum-of-products or products-of-sum has at least one minterm/maxterm

$$F = xy'z + xyz' + yz$$

- Non-standard: not in sum-of-product or product-of-sum format

$$F = x(y'z + yz') + yz$$

DeMorgan's Law

$$\begin{aligned}
 & \begin{array}{c} a \\ b \end{array} \xrightarrow{\text{OR}} F \quad F(a,b) = a+b = \Pi(0) \leftarrow \\
 & \overline{\overline{F}} = (a+\overline{b})(\overline{a}+b)(\overline{a}+\overline{b}) \\
 & \quad = \Pi(1,2,3) \\
 & \overline{\overline{\overline{F}}} = \overline{F} = \overline{(a+\overline{b})(\overline{a}+b)(\overline{a}+\overline{b})} \\
 & \quad = \overline{a+\overline{b}} + \overline{\overline{a}+b} + \overline{\overline{a}+\overline{b}} \\
 & \quad = \overline{a}b + a\overline{b} + ab = \Sigma(1,2,3) \leftarrow
 \end{aligned}$$

Boolean Algebra

OR



$F(a,b) = a + b = \prod(0)$

$$F = a(\bar{b} + b) + (\bar{a} + a)b$$

$$= a\bar{b} + ab + \bar{a}b + \cancel{ab}$$

$$= \bar{a}b + a\bar{b} + ab$$

$$= \sum(1, 2, 3)$$

Diagram illustrating the derivation of the OR function using Boolean algebra. The function is shown as $F(a,b) = a + b = \prod(0)$. The algebraic steps are: $F = a(\bar{b} + b) + (\bar{a} + a)b$, $= a\bar{b} + ab + \bar{a}b + \cancel{ab}$, $= \bar{a}b + a\bar{b} + ab$, and $= \sum(1, 2, 3)$. A red arrow connects the $\prod(0)$ term to the $\sum(1, 2, 3)$ term, indicating the relationship between the product of maxterms and the sum of minterms.

DeMorgan's Law

AND

$$\begin{matrix} a \\ b \end{matrix} \rightarrow \text{AND} \rightarrow F$$

$$F(a,b) = ab = \Sigma(3) \leftarrow$$

$$\overline{F} = \overline{a} \overline{b} + \overline{a} b + a \overline{b} = \Sigma(0,1,2)$$

$$\overline{\overline{F}} = F = \overline{\overline{a} \overline{b} + \overline{a} b + a \overline{b}}$$

$$= (\overline{\overline{a} \overline{b}})(\overline{\overline{a} b})(\overline{a \overline{b}})$$

$$= (a + b)(a + \overline{b})(\overline{a} + b)$$

$$= \Pi(0,1,2) \leftarrow$$

Boolean Algebra

AND

$a \text{ --- } \bigcap \text{ --- } b \text{ --- } F$

$F(a, b) = ab = \sum(3) \leftarrow$

$$F = ab + \bar{a}a + \bar{b}b$$

$$= a(b + \bar{a}) + \bar{b}b$$

$$= [a(b + \bar{a}) + \bar{b}][a(b + \bar{a}) + b]$$

$$= [(a + \bar{b})(b + \bar{a} + \bar{b})][(a + b)(b + \bar{a} + b)]$$

$$= (a + \bar{b})(1)(a + b)(\bar{a} + b)$$

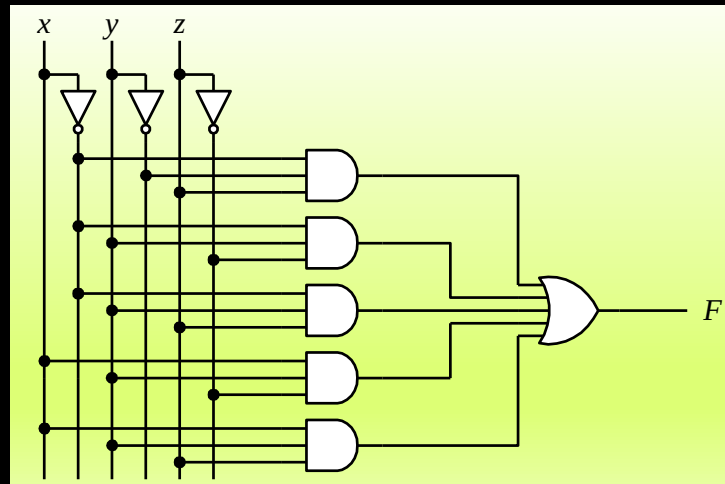
$$= (a + b)(a + \bar{b})(\bar{a} + b)$$

$$= \prod(0, 1, 2) \leftarrow$$

Quiz 2.3!

Digital Circuit

- Digital circuit is a connection of two or more logic gates
- Digital network can be described using schematic diagrams, Boolean expressions, or truth tables



$$F(x, y, z) = x'y'z + x'yz' + x'yz + xyz' + xyz$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Design a Car Security System

Designing a Car Security System

We will now go through the process of designing a small digital control system that we are all familiar with. In a car security system, we usually want to connect the siren in such a way that the siren will activate when it is triggered by one or more sensors. In addition, there will be a master switch to turn the system on or off. Let us assume that there is a car door switch D , a vibration detector switch V , and a master switch M . We will use the convention that when the door is opened, $D = 1$, otherwise, $D = 0$. Similarly, when the car is being shaken, $V = 1$, otherwise, $V = 0$. Thus, we want the siren S to turn on (i.e., set $S = 1$) when either $D = 1$ or $V = 1$ or when both $D = 1$ and $V = 1$, but only for when the system is turned on (i.e., when $M = 1$). However, when we turn off the system, we do not want the siren to turn on regardless of the state of the door switch or the vibration switch. Hence, when $M = 0$, it does not matter what values D and V have, the siren should remain off (i.e., set $S = 0$).

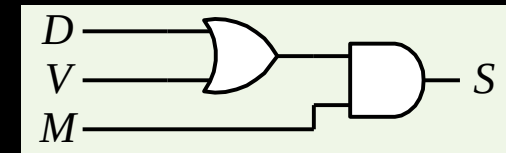
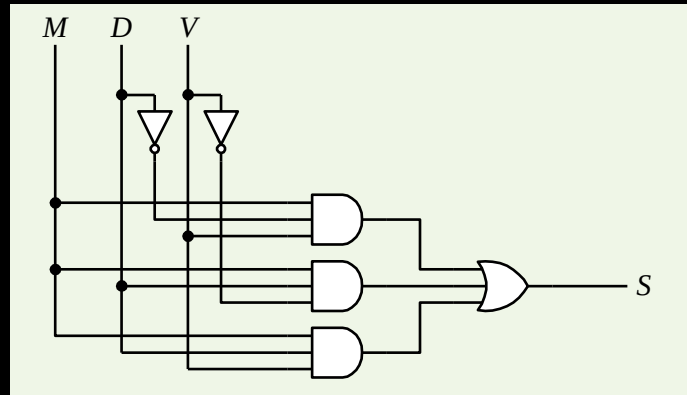
Given the above description of a car security system, we can build a digital circuit that meets our specifications. We start by constructing a truth table, which is basically

Design a Car Security System

- Input: D = Door switch
V = Vibration sensor
M = Motion sensor
Output: S = Siren

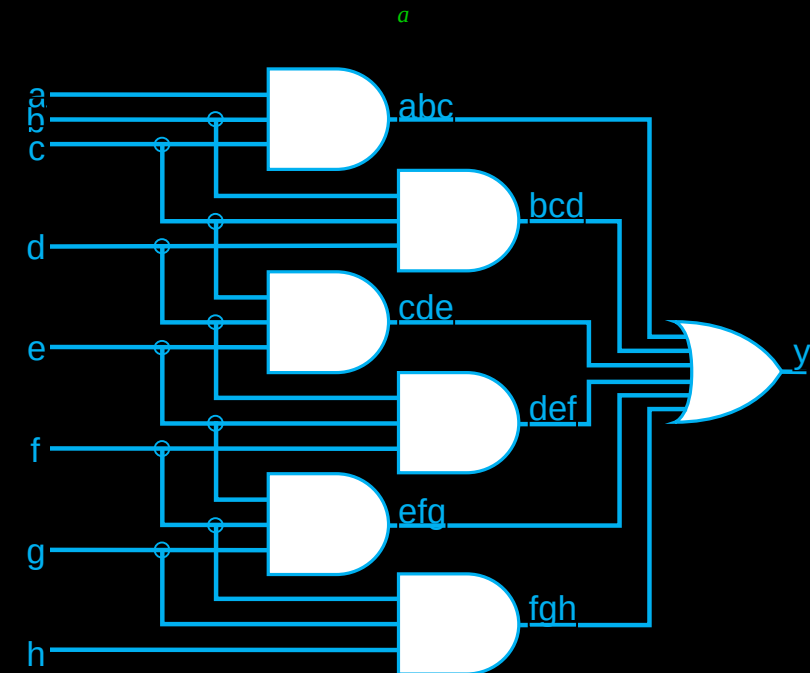
M	D	V	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$\begin{aligned}
 S &= (M D' V) + (M D V') + (M D V) \\
 &= M (D' V + D V' + D V) \\
 &= M (D' V + D V' + D V + D V) \\
 &= M (D(V' + V) + V(D' + D)) \\
 &= M (D(1) + V(1)) \\
 &= M (D + V)
 \end{aligned}$$



Example: Three 1s Pattern Detector

- Problem: Detect three consecutive 1s in 8-bit input: abcdefgh
 - 00011101 → 1
 - 10101011 → 0
 - 11110000 → 1
- **Step 1: Capture the function**
 - Truth table or equation?
 - Truth table too big: $2^8=256$ rows
 - Equation: create terms for each possible case of three consecutive 1s
 - $y = abc + bcd + cde + def + efg + fgh$
- **Step 2a: Create equation** -- already done
- **Step 2b: Implement** as a gate-based circuit



Example: Number of 1s Counter

- Problem: Output in binary on two outputs yz the # of 1s on three inputs

- $010 \rightarrow 01$

- $101 \rightarrow 10$

- $000 \rightarrow 00$

- **Step 1: Capture the function**

- Truth table or equation?

- Truth table is straightforward

- **Step 2a: Create equations**

- $y = a'bc + ab'c + abc' + abc$

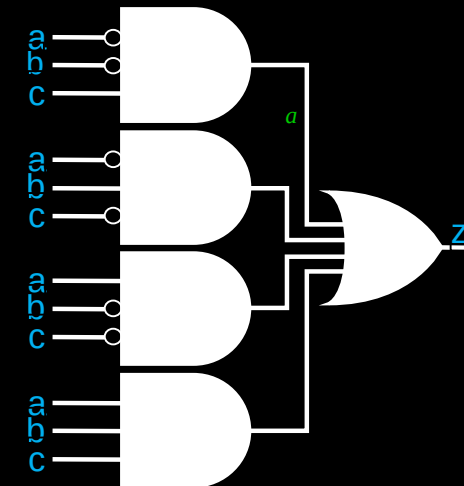
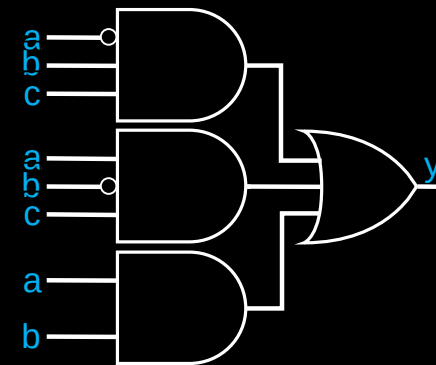
- $z = a'b'c + a'bc' + ab'c' + abc$

- Optional: Let's simplify y:

- $y = a'bc + ab'c + ab(c' + c) = a'bc + ab'c + ab$

- **Step 2b: Implement as a gate-based circuit**

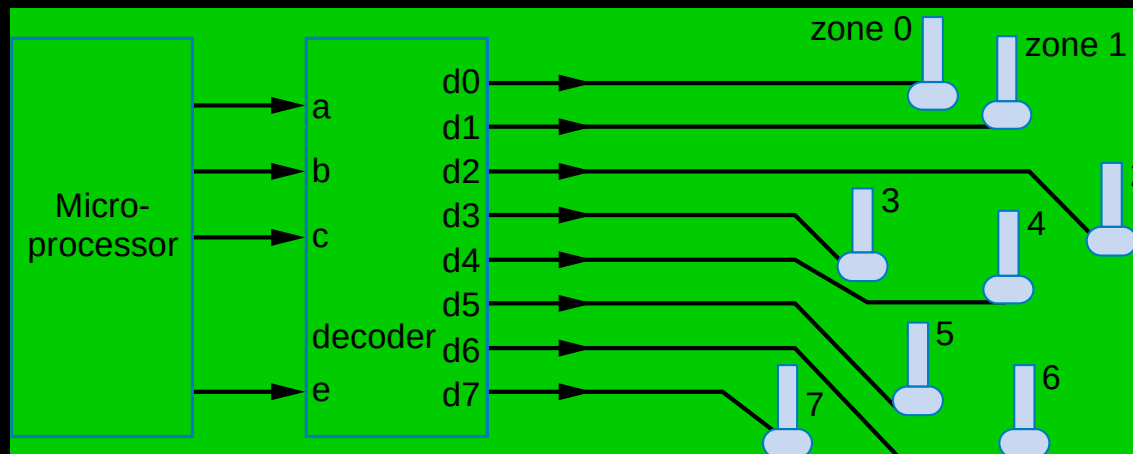
Inputs			# of 1s	Outputs	
a	b	c		y	z
0	0	0	(0)	0	0
0	0	1	(1)	0	1
0	1	0	(1)	0	1
0	1	1	(2)	1	0
1	0	0	(1)	0	1
1	0	1	(2)	1	0
1	1	0	(2)	1	0
1	1	1	(3)	1	1



Example: Sprinkler Controller

- Microprocessor outputs which zone to water (e.g., cba=110 means zone 6) and enables watering (e=1)
- Decoder should set appropriate valve to 1

Step 1: Capture
behavior



$$d0 = a'b'c'e$$

$$d1 = a'b'ce$$

$$d2 = a'bc'e$$

$$d3 = a'bce$$

$$d4 = ab'c'e$$

$$d5 = ab'ce$$

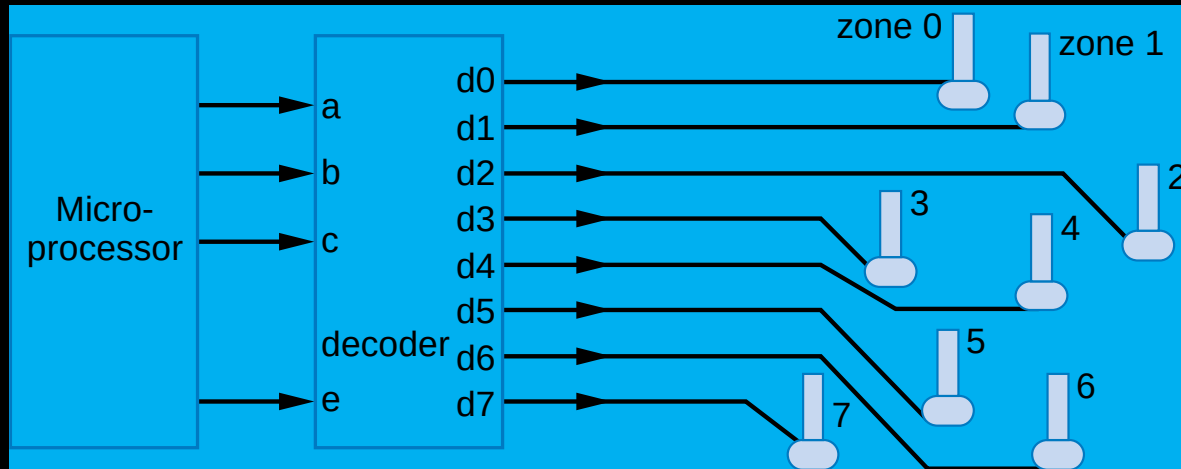
$$d6 = abc'e$$

$$d7 = abce$$

*Equations seem like
a natural fit*

Example: Sprinkler Controller

- Step 2b: Implement as circuit



$d0 = a'b'c'e$
 $d1 = a'b'ce$
 $d2 = a'bc'e$
 $d3 = a'bce$
 $d4 = ab'c'e$
 $d5 = ab'ce$
 $d6 = abc'e$
 $d7 = abce$

